1 Illustrative Examples

1.1 Examples of Generated Feedback

For illustration, we compare the responses generated by the compiler, a baseline model, few-shot PE technique, and the PEFT-model in order to analyze the effectiveness and quality of their feedback on a given code snippet. Consider the following code:

```
class Car {String model;
    public Car(String m) { model = m; }
    }
public class Garage {
    public static void main(String[] args) {
        Car c1 = new Car("Sedan");
        Car c2 = new Car(c1); }}
```

The compiler error message is "The constructor Car(Car) is undefined". This might be challenging for a novice programmer to understand as it requires a certain level of familiarity with programming fundamentals. The baseline model response is shown in Response 1.

Response 1: Baseline response

KM: The code is missing a constructor for the 'Car' class that takes no arguments. This is a common mistake that can lead to unexpected behavior when creating objects.
KH: To fix the issue with the missing constructor, add a constructor to the 'Car' class that takes no arguments. To fix the issue with the uninitialized 'model' field, initialize the field in the constructor.

Response 2: Few-shot response

KM: The code is attempting to create a copy of a Car object using the copy constructor, but it is not doing so correctly.
KH: The code should use the 'clone()' method to create a copy of the Car object, rather than attempting to create a new object with the same constructor.

Although the baseline model's Response 1 includes some explanation of the code, learners may find the response of limited usefulness. For feedback to be effective, it should clearly identify the core issue in the code and articulate it in a manner that facilitates the learner's comprehension. We also observed that

Response 3: PEFT-model response

KM: The copy constructor ‘Car(Car c)‘ is missing.
KH: Add a copy constructor ‘Car(Car c)‘ to perform a deep copy of the ‘model‘ field.

Response 4:

Ground truth from test set **KM:** ‘Car‘ has no copy constructor, so ‘new Car(c1)‘ is invalid. Constructors are not inherited.
KH: Define a copy constructor in ‘Car‘ to initialize a new object using an existing ‘Car‘ instance.

the baseline model's responses (although not present in this example) included generic comments about coding style and Java naming conventions, in addition to the feedback related to the code. In many cases, the corrected code was also provided along with the response. This can undermine the learning process, as learners would be discouraged from pursuing independent problem-solving and critical thinking, which are essential skills to master programming.

Although the few-shot prompt response identifies the issue in the code, it contains complex statements in the KH response as shown in Response 2. The fine-tuned model's response is shown in Response 3. In contrast to the baseline model, the fine-tuned model provides concise and relevant feedback and does not mention code styling and other irrelevant elements. The model also preserves the learning objective by not revealing the correct solution. The above response aligns closely with the expected ground truth for the code snippet, shown in Response 4.

1.2 Example of Sample Code and Feedback

The following is a sample code snippet along with Feedback E, C, F used in the focus group discussion.

```
class Animal {
    public void move() {
     System.out.println("Animal moves");
        }
        }
class Bird extends Animal {
  public void move(String location) {
   System.out.println("Bird flies to " + location);   }
         }
public class Main {
   public static void main(String[] args) {
        Animal myBird = new Bird();
        myBird.move("tree"); }}}
```

Feedback E: Compiler error message

The method move() in the type Animal is not applicable for the arguments (String)

Feedback C: Feedback from ChatGPT

The code has a compile-time error. What's wrong ?
Animal myBird = new Bird(); myBird.move("tree");

- The reference type is Animal.
- The class Animal defines only:
- public void move()
- It does not define:
- move(String)

So the compiler checks Animal and finds no move(String) method, even though the actual object is a Bird. Method calls are checked at compile time using the reference type, not the object type. Typical compiler error : The method move() in the type Animal is not applicable for the arguments (String) Or cannot find symbol: method move(String) Why this happens ?

- Bird does not override move()
- It overloads it by adding a different method:
- public void move(String location)

Overloading does not work through a superclass reference. How to fix it (any one of these)

- Option 1: Override move() correctly

```
class Bird extends Animal {
    @Override
    public void move() {
        System.out.println("Bird flies");}}
```

- Option 2: Add move(String) to Animal

```
class Animal {
    public void move(String location) {
        System.out.println("Animal moves to " + location);  }}
```

- Option 3: Change the reference type

```
Bird myBird = new Bird();
myBird.move("tree");
```

- Option 4: Cast (not recommended unless necessary)

```
((Bird) myBird).move("tree");
```

Feedback F: Feedback from fine-tuned model

Knowledge about Mistake: Reference type ‘Animal‘ has no ‘move(String)‘ method. Overloading resolution depends on the reference type (‘Animal‘), not the object type (‘Bird‘).

Knowledge about How to Proceed: Cast ‘myBird‘ to ‘Bird‘ to access the subclass method, or ensure the method exists in the reference type.